# Inteligencia artificial - Practica 1

[Elias Agustin Mihailov]
NIP: [927656]

# 1. Introducción y objetivos

Esta memoria recoge las conclusiones y decisiones de código de la práctica 1 de inteligencia artificial.

La práctica consta de dos partes, una primera de añadir métricas a los algoritmos de búsqueda, y ejecutar el problema del 8 puzzle con esos algoritmos, y una segunda donde se formula el problema de los caníbales y misioneros, representado en el código el problema, y luego mostrando una solución.

# 2. Métricas de los algoritmos

Para poder evaluar el rendimiento de cada algoritmo se han modificado los algoritmos y se han reescrito en searchEina.py de modo que cada función devuelva un diccionario de métricas con la siguiente información:

- profundidad: número de pasos a realizar hasta llegar a la solución encontrada.
- expandidos: número de nodos que se han expandido durante la búsqueda.
- generados: número total de nodos hijos generados.
- qSize: tamaño de la frontera en el momento en que termina la búsqueda.
- maxQS: tamaño máximo que alcanzó la frontera en algún momento de la ejecución
- tiempo: tiempo de ejecución en segundos

# 2.1. El problema del 8-puzzle: resultados experimentales

Se han definido 3 estado iniciales para ejecutar los algoritmos, y estos son estados del 8-puzzle cuya solución óptima se alcanza en 3, 9 y 30 pasos.

Para cada estado inicial se han ejecutado: búsqueda en anchura en grafo (BFS-G) y en árbol (BFS-T), búsqueda en profundidad en grafo (DFS-G) y en árbol (DFS-T), búsqueda en profundidad limitada con límite 3 (DLS-3) y límite 9 (DLS-9), profundización iterativa (IDS) y coste uniforme(en grafo) (UCS). Los casos que no finalizan en un tiempo razonable o agotan la memoria disponible se han marcado y comentado

## 2.2 Tabla de resultados

| Problema | Profundidad | Expand | Q.size | MaxQS | Tiempo | Generados |
|---|---|---|---|---|---|---|
| BFS-G-3 | 3 | 4 | 5 | 5 | 0.0000 | 11 |
| BFS-T-3 | 3 | 16 | 31 | 32 | 0.0010 | 48 |
| DFS-G-3 | 59123 | 120491 | 39830 | 42913 | 544.3517 | 160322 |
| DFS-T-3 | --- | --- | --- | --- | --- | --- |
| DLS-3-3 | 3 | 4 | 0 | 0 | 0.0000 | 12 |
| DLS-9-3 | 9 | 10 | 0 | 0 | 0.0000 | 27 |
| IDS-3 | 3 | 4 | 0 | 0 | 0.0010 | 12 |
| UCS-3 | 3 | 10 | 11 | 12 | 0.0000 | 22 |
| BFS-G-9 | 9 | 287 | 198 | 199 | 0.0060 | 487 |
| BFS-T-9 | 9 | 16877 | 29690 | 29691 | 0.0701 | 46568 |
| DFS-G-9 | 44665 | 141452 | 32012 | 42967 | 622.1456 | 173465 |
| DFS-T-9 | --- | --- | --- | --- | --- | --- |
| DLS-3-9 | LIMITE ALCANZADO | LIMITE ALCANZADO | LIMITE ALCANZADO | LIMITE ALCANZADO | 0.0000 | LIMITE ALCANZADO |
| DLS-9-9 | 9 | 5474 | 0 | 0 | 0.0403 | 15925 |
| IDS-9 | 9 | 5474 | 0 | 0 | 0.0560 | 15925 |
| UCS-9 | 9 | 486 | 276 | 278 | 0.0519 | 763 |
| BFS-G-30 | 30 | 181057 | 365 | 24048 | 368.0388 | 181424 |
| BFS-T-30 | --- | --- | --- | --- | --- | --- |
| DFS-G-30 | 62856 | 80569 | 41533 | 41534 | 342.1279 | 122103 |
| DFS-T-30 | --- | --- | --- | --- | --- | --- |
| DLS-3-30 | LIMITE ALCANZADO | LIMITE ALCANZADO | LIMITE ALCANZADO | LIMITE ALCANZADO | 0.0000 | LIMITE ALCANZADO |
| DLS-9-30 | LIMITE ALCANZADO | LIMITE ALCANZADO | LIMITE ALCANZADO | LIMITE ALCANZADO | 0.0280 | LIMITE ALCANZADO |
| IDS-30 | --- | --- | --- | --- | --- | --- |
| UCS-30 | 30 | 181423 | 16 | 24048 | 2633.8318 | 181440 |

**Notas:**

- El algoritmo DFS-T no acaba en ningún momento debido a bucles infinitos en el intento de encontrar la solución. Ya que no guarda en memoria los estados visitados, siempre vuelve a la misma disposición una y otra vez, nunca acabando la ejecución.
- En el algoritmo DLS, cuando pone LIMITE ALCANZADO, es que el algoritmo no ha podido encontrar una solución para el número de pasos indicado
- El IDS-30 no finaliza ejecución por el tiempo que requiere para acabar, ya que su complejidad temporal ha aumentado mucho y no guarda los estados visitados(visitar los 100 y pico mil estados no costaría tanto tiempo como el que tarda el IDS)
- El algoritmo BFS-T al tampoco guardar estado visitado, acaba expandiéndose cada vez más, hasta agotar la memoria disponible en un ordenador. No finaliza por falta de memoria

### 2.3 Análisis de los resultados

**Búsqueda en anchura (BFS-G vs. BFS-T).** La versión en grafo es muy superior a la versión en árbol: para el caso de profundidad 9, BFS-G expande 287 nodos frente a los 16 877 de BFS-T, y para profundidad 30 BFS-T directamente agota la memoria. Al descartar estados repetidos, la búsqueda en grafo evita en gran parte la complejidad espacial de su versión en árbol, aunque ambos acaban llegando a la misma solución con número de pasos.

**Búsqueda en profundidad (DFS-G vs. DFS-T).** DFS-T no finaliza en ninguno de los tres casos, al no recordar estados acaba con facilidad en bucles infinitos. DFS-G sí finaliza, pero de forma muy poco óptima: para una solución a solo 3 pasos, DFS-G encuentra una solución a profundidad 59 123, expandiendo más de 120 000 nodos(casi la totalidad de estados en el problema del 8-puzzle).

**Búsqueda en profundidad limitada (DLS).** Depende en gran medida del conocimiento del problema y los pasos a su solución, al utilizar un DFS-T internamente.

**Profundización iterativa (IDS).** Aprovecha el hecho de que el DLS con el número de pasos adecuado llega a una solución óptima, y recorre desde profundidad uno hasta un límite alto los algoritmos DLS con profundidad límite creciente.

**Coste uniforme (UCS).** Como todas las acciones del 8-puzzle tienen coste 1, UCS es como el BFS-G en cuanto al número de nodos expandidos y generados. Sin embargo, el tiempo de ejecución es mayor en UCS: 2633,8 segundos frente a 368,0 en BFS-G-30, a pesar de expandir un número de nodos similar.

## 3. El problema de los Misioneros y los Caníbales

### 3.1 Modelado del problema

Se ha implementado la clase Canibals, heredada de Problem con:

**Estado:** tupla (C_izq, M_izq, Bote, C_der, M_der), donde Bote vale 0 si el bote está en la orilla izquierda y 1 si está en la derecha, y el resto de elementos son el número de caníbales o misioneros. El estado inicial es (3, 3, 0, 0, 0) y el estado objetivo (0, 0, 1, 3, 3).

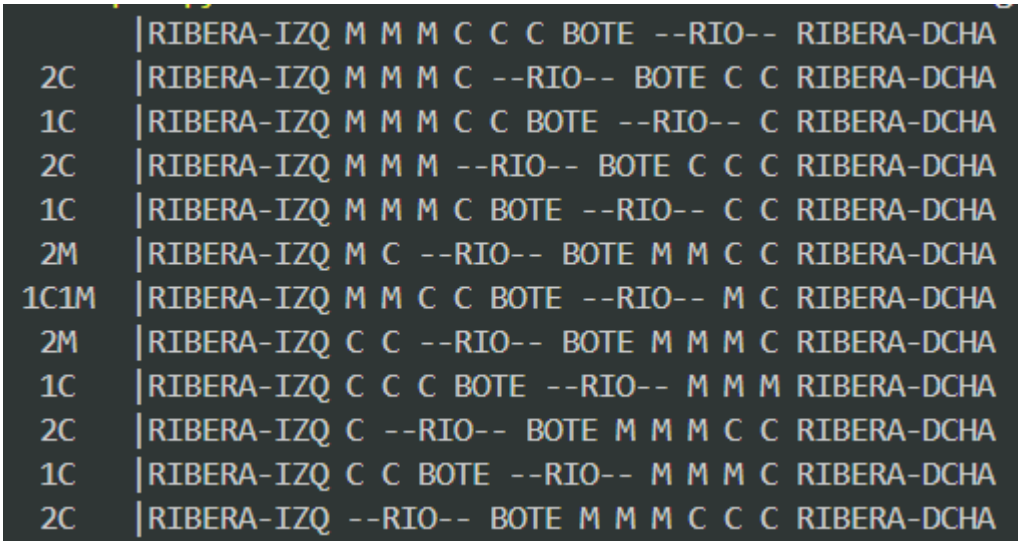
**Acciones:** ['1C', '2C', '1C1M', '1M', '2M'], que representan mover en el bote 1 caníbal, 2 caníbales, 1 caníbal y 1 misionero juntos, 1 misionero, o 2 misioneros, respectivamente, desde la orilla en la que se encuentra el bote hacia la otra.

**Función actions(state):** genera la lista de acciones posibles mediante el descarte de aquellas que no son aplicables: las que no tienen suficientes personas en la orilla de origen y las que dejarían que los misioneros fueran devorados.

**Función result(state, action):** Invierte el valor de Bote, y devuelve de nuevo el estado modificando los caníbales o misioneros desplazados.

**Función goal_test(state):** comprueba que el estado coincide con (0, 0, 1, 3, 3), es decir, que todos los caníbales y misioneros han cruzado a la orilla derecha.

### 3.2 Ejemplo de ejecución




### 3.3 Comentarios

Se ha usado el algoritmo BFS-G, ya que el problema al contar con pocos estados, una vez que se almacenan unos pocos se llega a la solución más rápidamente, aunque la complejidad sea la misma. Por el mismo motivo no se ha usado UCS, ya que todas las acciones tienen coste 1 y por tanto el BFS se comporta igual.

Además se han modelado las acciones posibles de manera que si el número de canibales y misioneros aumenta(siempre que el estado inicial sea uno válido), las acciones también puedan operar sobre ellos, no son unos ciertos estados fijos.